

Jupyter A11Y: A Study in Building Accessible Web Applications

James Martin

Chapel Hill
2016

Abstract

According to the World Health Organization, 285 million people are blind or visually impaired [1]. With the recent growth of jobs in technology sectors such as software development and data science, accessible tools are needed to enable the visually impaired to work in these fields. Technologies such as screen readers can be utilized to make existing tools more accessible but the practice of building accessible tools from the ground up is not common. However, the proliferation of the world wide web and the coincidental development of accessible web technologies make the web browser a great platform to build such tools. In this paper, we describe an extension to Project Jupyter, a web-based environment for interactive computing, that provides audio cues and screen reading capabilities far beyond traditional accessibility technologies. Further, we gather feedback from an impaired user to determine the effectiveness of our extension and to guide future work.

1. Introduction

Visually impaired users struggle to read text on the web. They frequently rely on magnification, which often leads to broken layouts and unexpected difficulties. For example, navigation bars and menus are unintentionally magnified along with text, unnecessarily taking up valuable screen space.

1.1 Web Accessibility Initiatives

Many traditional assistive technologies such as screen readers rely on the Hypertext Markup Language (HTML) of web pages to properly communicate content. The W3C group identifies four aspects of HTML that make accessible support of dynamically generated content difficult. To address these four concerns and to provide guidance for creating truly accessible web content, the W3C issued a specification titled Accessible Rich Internet Applications (ARIA). Quoting from the specification [6]:

1. Accessibility of dynamic content relies on abstracting semantics from both content and presentational information. Extracting semantic cues from current HTML content is typically unreliable as the cues are limited to tag element names.
2. While HTML allows content to be repurposed for presentational formatting, it lacks the ability to attach meaningful metadata about document structure and to convey semantic information. For example, some websites format content with tables rather than style sheets.
3. When combined with script and cascading style sheets (CSS), HTML can be repurposed to create dynamic custom components. However, dynamic content is generated without

providing a means to convey semantic information to native accessibility architectures designed to support such content.

4. Custom components built from common HTML elements often are not keyboard accessible.

The ARIA specification details a robust “ontology of roles, states, and properties that define accessible user interface elements” [6]. At its core, ARIA is a set of attributes that can be used to enhance existing HTML markup to help assistive technologies make sense of dynamically generated web content. Though the ARIA specification establishes a standardized method of making dynamic web content accessible, its adoption is not widespread. Jupyter Notebook does not use ARIA attributes.

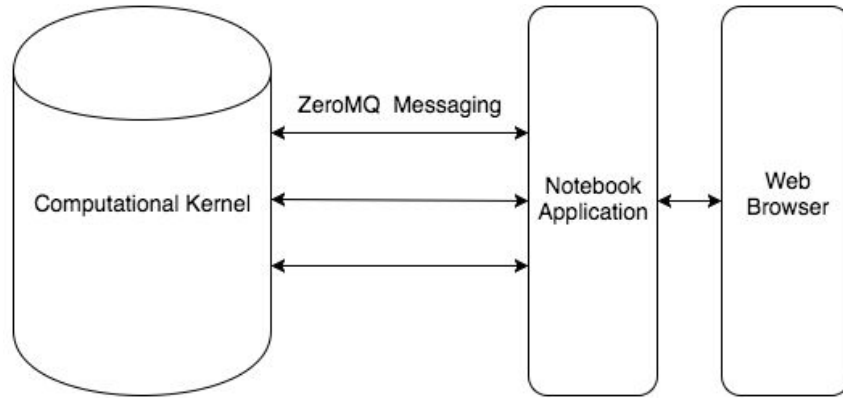
In addition to the ARIA specification, The Accessibility Project, also known as The A11Y Project, aims to improve the accessibility of the web through a philosophy of incremental change. By spreading awareness of the ARIA specification and sharing short blog posts on how to tweak websites to be more accessible, the A11Y Project hopes to make the web easier for impaired users to navigate.

We share the A11Y Project’s philosophy of incremental improvement and believe this model for change could be adopted to modify Jupyter to be a more accessible tool for hobbyists, developers, engineers, educators, students, and scientists.

1.2 Project Jupyter

Project Jupyter is a community-driven approach to provide an open source, interactive scientific computing environment to over 40 programming languages. Jupyter Notebook is a web application that allows users to create and share documents that contain live code, equations, visualizations, and explanatory text [10]. Useful for educators, researchers, and developers, Jupyter provides an intuitive interface to teach programming, explore data, and rapidly prototype code.

In summary, users start the Jupyter Notebook application, which opens in a browser of the user’s choice, and write code in “cells” to be executed by a computational server, called a kernel, upon the user’s request. Cells can include code, markdown, and plain text [13]. After execution, output is sent back to the Notebook to be rendered in the browser. Using the browser for the Notebook client allows the flexibility to render content using HTML, CSS, and JavaScript. These technologies also allow for third-parties to create their own additions to the Notebook interface.



Jupyter makes use of a client-server architecture to run and display code, graphs, markdown, and text in the Notebook interface. Communication with the Notebook client and kernel server takes place over an open network protocol. As mentioned, Jupyter supports computing on over 40 programming languages, and popular kernel implementations include Python, R, Scala, and Julia. The technical details of the network protocol and computational kernels are not discussed in this paper, as our primary focus is on the client-facing Notebook, which is agnostic to the language of the kernel [10].

In the Notebook interface, cells are edited using CodeMirror, an in-browser text editor that supports a myriad of features such as customizable syntax highlighting [5]. Much of the information about the text contained in Notebook cells is accessible through the CodeMirror API, which a developer can inspect by probing elements of the Notebook JavaScript object in the browser.

1.3 Our contribution

Jupyter Notebook’s dynamically generated content has very limited accessibility. The HTML of the rendered web page of the Notebook does not implement the ARIA specification. However, the Notebook maintains information about the context of each cell. This information is also stored in the Document Object Model (DOM) of the webpage, which can be read and modified to benefit assistive technologies such as screen readers.

In what follows, we propose and detail a JavaScript extension to the Jupyter Notebook, nbreader, which can be installed using the Notebook application itself to aid navigation of the interface, and provide additional context to the user through sound and speech.

2. Prior Work

In addition to ARIA specifications from the W3C group, other researchers and developers have made significant contributions to making technology more accessible. Since computers communicate information visually, users with visual impairments must rely on

assistive technologies that communicate information aurally. The introduction of the *earcon*, described below, was a large step in providing context using sound. Further, the development of tools such as Emacspeak, an extension to the Emacs editor, and screen readers like Job Access With Speech (JAWS) are two examples of a greater effort to adapt computers to the needs of users with visual impairments. Lastly, several application programming interfaces (APIs) are available to web developers to add sound and speech to web sites and web applications.

2.1 Earcons

Navigation of computer interfaces changed greatly in the early 1970s with the introduction of graphical symbols known as icons. Icons can represent system state, programs, cursors, menus and menu items, as well as objects such as files and documents [4]. In this fashion, icons can present recognizable visual information.

Earcons aim to present information as quickly as icons do, but with structured sound. Blattner et al., discuss varying rhythm, pitch, timbre, and length, among others, to encode information. An example of these variations in practice is the low, baritone alert heard in many operating systems when an action attempted by the user cannot be performed.

Ambient sound can also provide valuable contextual information. Playing or looping extended earcons can signal the progress of a process over time. This can be especially useful for monitoring the completion of long computations. [11]

2.2 Emacspeak and Job Access With Speech (JAWS)

Two examples of existing assistive technologies are Emacspeak and Job Access With Speech (JAWS). JAWS is a popular screen reader developed for blind and visually impaired computer users. JAWS targets specific business productivity applications but supports ARIA markup, as well, to facilitate web browsing. Though it is a powerful enabling technology, JAWS provides an additional application layer rather than an integrated accessibility solution. For example, JAWS is limited when reading code in a development environment and has no understanding of indentation. By contrast, Emacspeak is an example of an integrated solution.

Emacspeak is a speech interface built on top of the extensible text editor Emacs. Dubbed “the complete audio desktop,” Emacspeak allows visually impaired users to enjoy the full suite of tools available in the Emacs environment through sound and speech. Developed by T.V. Raman, a blind computer scientist, Emacspeak is an example of an assistive technology built from the bottom-up to fully addresses the needs of its users [12].

To synthesize speech and sound, Emacspeak makes use of “hooks,” which are a mechanism for customizing Emacs. Hooks are variables in Lisp which hold a list of functions to be executed on a “well-defined occasion” [9]. Emacspeak wraps many occasions with functions, so that audio cues and speech are triggered when users execute commands or type text. Hooks also allow for developers to extend and customize Emacspeak to suit their individual needs.

Jupyter uses a similar model to update the Notebook upon state changes of the kernel. These state changes trigger events, which are “listened for” by the Notebook. Listeners for events triggered by the kernel function in the same way as hooks. Our contribution draws inspiration from Emacs hooks and performs speech and sound actions when events are triggered.

2.3 Accessibility through web APIs

WebSpeech API

Authored by Google and still an experimental technology, the WebSpeech API provides web developers with the capability to synthesize text to speech. The API can be used to retrieve information about available voices on the device, as well as start and pause speech [8]. Since Jupyter Notebook runs in the browser, we leverage the WebSpeech API to convert text to speech quickly and efficiently.

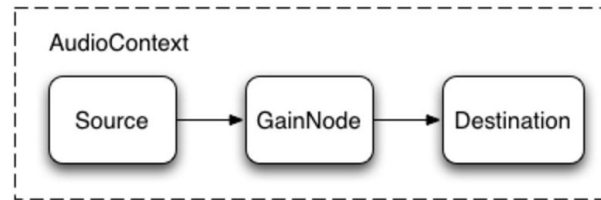
WebAudio API

The WebAudio API is a “high-level JavaScript API for processing and synthesizing audio in web applications” [2]. Though audio in the web has previously been delivered through plugins such as Flash or QuickTime, the WebAudio API allows developers to generate audio directly with JavaScript embedded in the web page. To generate sounds, developers create nodes that can be chained together with modular routing to create a range of sounds similar to those produced by a synthesizer instrument using familiar elements such as gain and delay.

An “AudioContext” is created to control signals sent to the users. The AudioContext maintains a graph of audio nodes and keeps an AudioDestinationNode, which is used to route signals to output devices that are typically hardware end-points. Only one AudioContext is needed per application.

The API takes a “fire-and-forget approach” to audio source scheduling, which works well for our use case. Source nodes for generating sound are created for each note during the lifetime of the AudioContext. Though the nodes the developer creates are never explicitly removed from the graph of the AudioContext, sound nodes cannot be played again once they have been stopped. A better approach to reusing sounds is to make use of a gain node to turn volume up and down. [2]

An example of a simple audio graph is shown below.



Audio graph with a gain node

3. Design

We model our extension to Jupyter Notebook after Emacspeak. Two crucial features of Emacspeak are reading highlighted text and audio cues upon key presses. Character keystrokes are read aloud to users, and cues are played for non-alphanumeric keys such as backspace. Cues are also played to help with navigation of Emacs, such as when new applications are opened, the user switches between buffers, or when errors occur.

To implement these features similar to Emacspeak, we bind listeners to three different sources to generate speech and sounds.

3.1 Kernel Events

Examples of audio cues that are generated from kernel events are detailed below. In the appendix, we detail an entire list of kernel events to which listeners can be bound.

Navigation

- nbreader is successfully loaded
 - A message is read to the user alerting them the extension is ready
- User changes the mode of the notebook (Command or Edit)
 - Jupyter Notebook uses vi/vim style modes of Command and Edit
 - Whenever the mode is switched in the notebook, the change is read to the user
- User issues a keyboard shortcut to read the current notebook mode
- User changes the selected cell
 - nbreader speaks whether the user has moved up or down a cell in the notebook
- User changes the selected cell and reaches the top / bottom of the page
 - nbreader speaks “top” when the user reaches the top cell and “bottom” when the user reaches the bottom cell

Other kernel events

- Kernel state changes to be busy when a cell is running
 - A low tone is played to indicate when the kernel is busy running a cell
- The current notebook is saved successfully
 - A message is read to the user indicating the notebook has been saved

3.2 CodeMirror API

The following functions of nbreader make use of the CodeMirror API. As mentioned in subsection 1.3, CodeMirror provides the in-browser code editor for Jupyter Notebook.

Reading code

- User issues a keyboard shortcut to read the currently selected cell
 - nbreader uses the CodeMirror API to extract code and create messages that are queued and read by the WebSpeech API
- User issues a keyboard shortcut to skip the current line being read
 - The aforementioned queuing system allows the user to skip the current line

3.3 Browser Events

We also bind listeners to events taking place in the browser to provide some functionality similar to Emacspeak.

Editing a cell

- User types a character while editing a cell
 - Like Emacspeak, nbreader verbalizes keystrokes
 - Alphanumeric characters can be read as normal, but characters such as slashes, braces, brackets, parentheses, periods, and operators are converted to string messages to be read by the WebSpeech API

Modifying the CSS of the page

- User wants to use magnification to view text in cells more easily
 - The user can toggle an “accessibility mode” to remove the toolbar from the interface, enabling much cleaner magnification of the page.

By implementing these features, we show creating a domain specific screen reader is not only possible but better suited for communicating helpful contextual audio cues. In the following section, we detail a formative study used to gather feedback on the extension and to direct future work.

4. Formative Study

The baseline for comparison of the usefulness of the nbreader extension was its ease of use compared to JAWS, mentioned in section 2.2. We asked the participant to answer with a brief response and compare nbreader to JAWS with a plus/neutral/minus scale.

Karthik Balasubramanian, Harvard Business School Class of 2018

Karthik is a scientist and entrepreneur with macular degeneration. While his peripheral vision is unaffected, his central vision has degraded to the point where he must magnify text to the largest possible size on his 15" laptop to read effectively.

To perform the study, we walked Karthik through installing Jupyter and the nbreader extension. We next ran cells of code in a sample notebook. Karthik also took over the keyboard and practiced navigating around the notebook. We also explained each keyboard shortcut to Karthik and allowed him to experiment with writing and reading code and text.

Ease of use questions

1. *How easy or difficult was the nbreader extension to install?* (Answer: +)

Karthik commented that nbreader would be a good tool to help beginners learn to program, as Jupyter Notebook itself is easy to install. He was also impressed by the ease of installation of nbreader, running two cells in a new notebook was very easy compared to other environments he has set up in the past.

2. *How easy or difficult is it to navigate the notebook environment with nbreader?* (Answer: 0)

Long sections of output caused some confusion while navigating the notebook. While Karthik thought moving up and down through the notebook was easy, he believed the audio cues noting the change of the selected cell felt unnecessary. He instead suggested that we include a new command to speak which cell in the notebook is selected (i.e. "in code cell 3 of 10").

3. *How easy or difficult is it to edit a cell with nbreader?* (Answer: +)

Karthik believed it was easy to switch between edit and command mode, making editing a cell easy. He liked how nbreader mimicked Emacspeak by reading characters and commented the feature helps cut down on typos.

4. *How easy or difficult is it to read the current cell you are using with nbreader?* (Answer: +)

The greatest asset to nbreader is building in functionality to read code. Karthik stated that JAWS is "useless" in a development environment, and the integrated screen reading feature was the most critical feature. He appreciated the commands to skip and cancel reading, as well.

5. *How easy or difficult are the keyboard shortcuts to use?* (Answer: 0)

Finally, Karthik thought the keyboard shortcuts were straightforward to use, but like any tool, would require practice and muscle memory.

After answering these questions, we prompted him to elaborate on any features he liked or did not like. We also asked him if there are any features that were blatantly missing or he would like to see implemented.

Formative questions:

1. Could you get an idea of the layout of the notebook with reader?
2. What are some features you found useful?
3. What are some features you were confused by?
4. What is a feature you would like to see implemented?

Feedback from formative questions

Karthik mainly had difficulty getting an idea of the layout of the notebook. He suggested four improvements to fix this difficulty. First, he suggested we add a bold, contrasting border around the selected cell. Second, as mentioned, he suggested we include a command to speak which cell in the notebook is selected. An observation we made that would facilitate Karthik's interactions with the notebook would be to force all output cells to have a fixed height. This way, long sections of output could be scrolled through or skipped. Additionally, Karthik's third suggestion was to separate input and output cells to facilitate reading and navigating. The final suggestion Karthik made was to implement a shortcut to collapse and reopen notebook cells. Working with R, Karthik discussed how he often does not have to touch previous segments of his code. Being able to collapse cells would shorten the notebook page and lessen confusion about the location of the user in the notebook.

5. Future work

Suggested improvements

From the feedback given to us by Karthik, we decided to immediately implement two features. First, we modified the styling of the notebook to include a bold green border around the currently selected cell. Second, we implemented the command to speak the position of the current cell.

Improvements to our contribution

Using a native text to speech engine would allow for greater flexibility in changing volume, pitch, and rate of the speech. Being able to modify these properties of native speech engines would enable integrated screen readers such as ours to synthesize text in accordance with other semantic information. The CodeMirror API provides our extension to Jupyter with syntax highlighted markup. This markup, along with a native text to speech engine, would enable us to implement screen reading that varies with syntax highlighting, providing additional context to visually impaired users.

A second improvement to our extension would be to improve the CSS used to style Jupyter Notebook. While we already modify the CSS of the interface to improve magnification, we also could make changes to the colors of the background and syntax highlighting to boost

contrast. Changing the CodeMirror cursor to be bigger and bolder would also help impaired users identify where they are editing.

Third, we could improve the timing of when screen reading is performed, especially in reaction to keypress events. We could build our extension to further mimic Emacspeak. For example, when the user is typing in Emacspeak and hits the spacebar after typing characters, the word the characters compose is read to the user. This immediately alerts the user to typos or mistakes in writing text or code. Combined with the feature of reading text with syntax highlighting detailed above, this feature could be a powerful tool for enabling visually impaired users to write code more efficiently.

Recommendations for building accessible applications

Client-server architectures and publisher-subscriber models make building accessibility into applications fairly straightforward. As long as an application has some method of listening for events, and some method of handling those events, the application can generate helpful auditory or tactile feedback to the user. Text-to-speech APIs provide web developers with a quick and easy way to synthesize speech from their applications. Natively, eSpeak is an open source solution to providing text-to-speech in applications for Linux and Windows [7]. Apple has their own specification and provide suggestions and strategies for incorporating synthesized speech into Mac applications [3].

Community and accessibility

Building accessible applications are often community-driven efforts. We hope that the visibility and growth of the Jupyter Project sheds some light on our contribution. Further, since we documented the process of building this extension and our findings on how the Notebook interface interacts with the kernel, we hope that the community can build upon our proof-of-concept. We also hope that this paper and our extension can provide a model for other developers working to build accessible applications.

6. Conclusions

From our experience integrating accessibility into an existing application, we found that our extension could communicate contextual information more effectively than a traditional screen reader. Though adding accessibility during the development process may be daunting, accessible applications attract a more inclusive user base. Enabling even the simplest of cues for impaired users is better than no accessibility at all, and such cues can facilitate user experiences in ways traditional screen readers cannot. Another route to making applications more accessible is to use an architecture that uses hooks or event listeners, such as a publisher-subscriber model. These architectures allow for individuals or communities to add integrated accessibility solutions.

A community driven approach to building accessible tools is an efficient way to develop and gather feedback. Furthermore, formative and user studies must be completed to direct these efforts. As most developers are not impaired, including user studies and having testers with impairments is the best way to ensure tools and applications are truly accessible.

7. Further reading

To view documentation of our extension and more information about the development process of nbreader, visit the GitHub page here: <http://jameslmartin.github.io/jupyter-a11y/>. On the webpage, one can find details on how to install and use the extension as well as a list of kernel events to which listeners may be bound.

References

- [1] The Accessibility Project. (2016). The A11Y Project. Retrieved April 28, 2016, from <http://a11yproject.com/>
- [2] Adenot, P., & Wilson, C. (2015, December 8). Web Audio API [Program documentation]. Retrieved April 28, 2016, from <https://www.w3.org/TR/webaudio/>
- [3] Apple. (2006, September 5). Introduction to Speech Synthesis. *Speech Synthesis Programming Guide*. Retrieved April 28, 2016, from <https://developer.apple.com/library/mac/documentation/UserExperience/Conceptual/SpeechSynthesisProgrammingGuide/Introduction/Introduction.html>
- [4] Blattner, M. M., Sumikawa, D. A., & Greenberg, R. M. (1989). Earcons and Icons: Their Structure and Common Design Principles. *Human-Computer Interaction*, 4, 11-44. Retrieved April 28, 2016.
- [5] CodeMirror API. (n.d.). Retrieved April 28, 2016, from <https://codemirror.net/doc/manual.html>
- [6] Craig, J., & Cooper, M. (Eds.). (2014). Accessible Rich Internet Application (WAI-ARIA) 1.0. *W3C Recommendation*. Retrieved April 28, 2016, from <https://www.w3.org/TR/wai-aria/>.
- [7] eSpeak text to speech. Retrieved April 28, 2016, from <http://espeak.sourceforge.net/index.html>
- [8] Google. Voice Driven Web Apps: Introduction to the Web Speech API. Retrieved April 28, 2016, from <https://goo.gl/RZ4P1q>
- [9] GNU. Emacs (Version 24.5) [Program documentation]. Retrieved April 28, 2016,
- [10] The Jupyter Project. (2016). Project Jupyter. Retrieved April 28, 2016, from <http://jupyter.org/>
- [11] Parente, P. (2008). *Clique: Perceptually Based, Task Oriented Auditory Display for GUI Applications* (Unpublished doctoral dissertation). University of North Carolina at Chapel Hill.

[12] Raman, T.V. On The Internet. Retrieved April 28, 2016, from
<http://emacspeak.sourceforge.net/raman/>

[13] Strobeck, K. What Is Markdown? Retrieved April 28, 2016, from
<http://whatismarkdown.com/>